Week 1 · Day 3: Transformers and LLMs

AI Intern Onboarding · Daily Task Sheet · Wednesday, 7 October 2026

Today's goal: understand what happens inside a large language model, from the moment text is split into tokens to the moment a word is sampled, and be able to explain in your own words why an LLM can sound confident and still be wrong. This is the theory behind the Week 1 deliverable: tomorrow you call an LLM to turn a sample visit note into a validated JSON summary, and today explains why we keep temperature low, validate every output and keep a clinician in the loop.

Data rule: use only public, de-identified or synthetic data today (and every day). No real patient information (PHI) in any exercise, notebook, prompt or daily note. Every sentence you tokenise or prompt with today must be made up by you.

Schedule (shift 2:00 pm to 11:00 pm)

Time	Block	Activity
14:00–14:15	Setup	Open Jupyter, create a *week1-day3* notebook, check that `transformers` and `torch` import and that a small model downloads.
14:15–17:45	Study (3.5 h)	Topics 1–5 below, in order. Take short notes as you go; about 40 min per topic.
17:45–18:30	Meal break	
18:30–21:30	Hands-on (3 h)	Practice task: tokenisation, attention and temperature experiments (steps below).
21:30–22:30	Discussion (1 h)	Explain today's ideas to your peers and mentor (prompts below).
22:30–23:00	Daily note (0.5 h)	One page in your own words (template below).

Study topics

1. Tokens: how text becomes numbers

- **What:** a model never sees letters or words; it sees token IDs. A tokenizer splits text into pieces (whole words, sub-words or single characters) from a fixed vocabulary, then maps each piece to a number. Common schemes: WordPiece (BERT) and byte-pair encoding, BPE (GPT). Each token ID is then turned into an embedding vector, the idea you met yesterday.
- **Why it matters:** cost, speed and the context limit are all counted in tokens, not words. Rare medical terms (drug names, Latin anatomy) often split into many fragments, so clinical text uses more tokens and the model may understand rare terms less well.
- **Do:** guess how many tokens "hydrochlorothiazide" becomes, then check it in the hands-on task.

2. Attention: letting every word look at every other word

- **What:** for each token the model asks "which other tokens matter for understanding me?" and builds a weighted mix of them. In "The tablet was too large, so the patient could not swallow it", attention helps *it* link to *tablet*. Each token makes a *query*, every token offers a *key* and a *value*; query–key similarity decides the weights. Many attention *heads* run in parallel, and layers are stacked.
- **Why it matters:** before transformers, models read text one word at a time (RNNs) and forgot early words. Attention sees the whole sequence at once and trains in parallel on GPUs, which made today's large models possible.
- **Where it breaks:** cost grows with the square of the sequence length, which is one reason context windows are limited.
- **Do:** draw arrows between the words of one sentence showing which words you think attend to which.

3. Encoder vs decoder models (BERT vs GPT)

- **Encoder (BERT):** reads the whole text in both directions at once; trained by filling in masked words. Good at *understanding*: classification, search, extracting entities.
- **Decoder (GPT):** reads left to right and predicts the next token, one at a time. Good at *generating*: chat, summaries, drafting. Encoder–decoder models (e.g. T5) do both, as in translation.
- **Why it matters:** our Note Summariser uses a decoder (chat) model; a search or coding-suggestion tool might use an encoder.
- **Do:** for each type, write one healthcare task you would use it for, and why.

4. How a chat model is made: pretraining, instruction tuning, human feedback

- **Pretraining:** next-token prediction on huge amounts of text. The result knows language and many facts but just continues text; it does not follow instructions.
- **Instruction tuning:** further training on example prompts and good answers, so it learns to respond helpfully.
- **Human feedback (RLHF and similar):** people rank answers and the model is tuned toward preferred ones (helpful, honest, harmless).
- **Where it breaks:** each stage teaches what *sounds* good. Nothing in the process guarantees that every statement is checked against a source.
- **Do:** draw the three stages as a pipeline and write what goes in and what comes out of each.

5. Temperature, context window and hallucination

- **Temperature:** at each step the model produces a probability for every possible next token. Low temperature (near 0) almost always picks the most likely token: repeatable, plain. High temperature flattens the odds: varied, creative and more error-prone. Top-k and top-p limit the choice to the likeliest tokens.
- **Context window:** the maximum number of tokens (prompt plus answer) the model can see at once. Anything outside it simply does not exist for the model; very long inputs can also make it miss details in the middle.
- **Hallucination:** fluent, confident text that is false or not supported by the input (e.g. a medication that is not in the note). It happens because the model optimises for plausible text, not verified truth.
- **Why it matters in healthcare:** a made-up dose or diagnosis in a summary can harm a patient. That is why we use low temperature, ask for structured output, validate it and keep a clinician reviewing.
- **Do:** list three ways you could reduce hallucination in the Note Summariser.

Hands-on practice: tokens, attention and temperature

Models: small, public, pretrained models from the Hugging Face Hub that run on a laptop CPU, such as `bert-base-uncased`, `gpt2` or `distilgpt2`, and a small instruction-tuned model such as `Qwen/Qwen2.5-0.5B-Instruct`. **Text:** 5 to 10 short sentences you invent yourself, e.g. "Patient reports dyspnoea on exertion; started on metoprolol and furosemide." No names, dates or details from real notes. Check each model's licence on its model card.

#	Step	Time
1	Load the tokenizers for a BERT model and a GPT-2 model with `AutoTokenizer`. Tokenise your made-up sentences; print the tokens, the IDs and the token count for each.	25 min
2	Compare how each tokenizer splits medical terms vs everyday words. Make a small pandas table: word, tokens (BERT), tokens (GPT-2). Note which terms split the most.	25 min
3	Decode the IDs back to text with `tokenizer.decode` and confirm you get your sentence back. Note anything that changes (case, spacing).	10 min
4	Run BERT with `output_attentions=True` on the "tablet ... swallow it" sentence. Plot one head's attention as a heatmap and look at which tokens *it* attends to. Try two or three heads.	35 min

#	Step	Time
5	Encoder vs decoder: use a `fill-mask` pipeline with BERT and a `text-generation` pipeline with GPT-2 on related prompts. Describe the difference in what each does.	20 min
6	Run the small instruct model on the same prompt (e.g. "Summarise this note in one sentence: ...") 5 times at temperature 0.1 and 5 times at 1.2. Compare how much the answers vary and whether any add facts not in the note.	35 min
7	Hallucination probe: ask the model about a drug you invent (e.g. "What is the usual dose of Zentravolin?"). Record whether it admits it does not know or confidently makes something up. Write your explanation in a markdown cell.	20 min
8	Tidy the notebook so it reruns top to bottom; save it.	10 min

Stretch (if time): use `max_new_tokens` and a long prompt to see what happens near the context limit; try top-k and top-p sampling and compare with temperature alone; print the top 10 next-token probabilities for one prompt and watch them change with temperature.

Challenge project: Build a tiny GPT and audit a small LLM for hallucination

Difficulty: hard. Start it after the core practice task; expect roughly 8 to 10 extra hours, so work on it in spare slots across Week 1 and present it on Friday alongside your summariser. Ask your mentor early if you get stuck.

Scenario: before the team trusts an LLM to summarise visit notes, they want someone who understands the machinery from the inside: how text becomes tokens, how attention and sampling work, and how often a small model makes things up. Build the core pieces yourself, then measure a real model's failure modes and report them honestly.

Data: public text only (e.g. the Tiny Shakespeare text used in many tutorials) and question sets and notes you write yourself. Facts for the question set should come from public references. Never use real clinical records.

Milestones

#	Milestone	What to deliver
1	Tokenizer audit	Tokenise a list of 50 medical terms you compile from public sources with three tokenizers (BERT, GPT-2 and one biomedical model's tokenizer from the Hub). Report average tokens per term and the worst splits. Explain what this means for cost and context length.
2	Attention from scratch	Implement scaled dot-product attention with a causal mask in PyTorch. Check your output against `torch.nn.functional.scaled_dot_product_attention` on random inputs. Explain in one paragraph why the scores are divided by the square root of the key size.
3	Tiny decoder model	Build a small character-level GPT (embeddings, positional embeddings, multi-head attention, feed-forward, layer norm) and train it on a public text. Plot training and validation loss; apply yesterday's overfitting checks.
4	Sampling, your own way	Implement greedy, temperature, top-k and top-p sampling yourself. Generate from your model at several settings and measure repetition (e.g. share of repeated n-grams) against temperature.
5	Encoder vs decoder	On a small labelled set of made-up sentences (e.g. note sentence: symptom / medication / follow-up), compare BERT embeddings plus logistic regression with prompting a small decoder model. Report precision, recall and F1 for both, as on Day 1.

#	Milestone	What to deliver
6	Hallucination audit	Write 30 questions with known answers from public references and 10 unanswerable ones (invented drugs, impossible facts). Run a small instruct model at two temperatures. Label each answer correct, wrong or abstained, and report rates with bootstrap confidence intervals.
7	Confidence vs correctness	Record the average token log-probability of each answer. Does higher model confidence predict correctness? Plot it, and link it to Day 1's calibration ideas.
8	Context test and report	Hide one fact at different positions in long made-up notes and measure how often the model finds it by position and length. Finish with a 1–2 page markdown report: findings, limitations, and concrete rules for the Note Summariser (temperature, validation, clinician review).

Success criteria

- Notebook or repo reruns end to end from a fresh environment (requirements file included; seeds fixed).
- Your attention function matches PyTorch's to a stated tolerance.
- Every claim about model behaviour has a number, a sample size and an uncertainty estimate.
- The report states plainly where the model should not be trusted and what safeguards follow from that.
- You can defend each design choice in a 10-minute walkthrough.

Stretch goals

- Train your own small BPE tokenizer on public text and compare its splits of medical terms with GPT-2's.
- Add a key-value cache to your tiny GPT and measure the generation speed-up.
- Re-run the hallucination audit with an instruction in the prompt telling the model to say "I don't know" when unsure, and measure whether abstention improves.
- Visualise attention in your own model across layers for one sentence.

Resources to study from

Core (free)

Resource	Use it for
Hugging Face LLM Course (huggingface.co/learn)	Hands-on chapters on transformers, tokenizers and pipelines; the closest match to today's practice task.
Hugging Face Transformers documentation (huggingface.co/docs/transformers)	Official reference for `AutoTokenizer`, `AutoModel`, pipelines and generation settings (temperature, top-k, top-p).
Jay Alammar, "The Illustrated Transformer" and "The Illustrated GPT-2" (jalammar.github.io)	Picture-by-picture walkthroughs of attention, encoders and decoders.
3Blue1Brown, Neural Networks series: chapters on transformers and attention (YouTube)	Visual intuition for embeddings, attention and next-token prediction.
Andrej Karpathy, "Let's build GPT" and "Let's build the GPT Tokenizer" (YouTube)	Build a small GPT and a BPE tokenizer step by step; the backbone of the challenge project.
Andrej Karpathy, "Intro to Large Language Models" (YouTube)	One-hour overview of pretraining, fine-tuning, human feedback and LLM limits.

Deeper reading

- Vaswani et al., 'Attention Is All You Need' (2017): the original transformer paper; read the introduction and Section 3.

- Devlin et al., 'BERT' (2019) and Radford et al., 'Language Models are Unsupervised Multitask Learners' (GPT-2, 2019): encoder vs decoder in practice.
- Ouyang et al., 'Training language models to follow instructions with human feedback' (2022): how a chat model is made.
- Holtzman et al., 'The Curious Case of Neural Text Degeneration' (2020): why sampling settings matter; introduces top-p.
- Liu et al., 'Lost in the Middle: How Language Models Use Long Contexts' (2023): background for milestone 8.
- Ji et al., 'Survey of Hallucination in Natural Language Generation' (2023): types and causes of hallucination.
- Sebastian Raschka, *Build a Large Language Model (From Scratch)*: chapters on attention and GPT architecture.
- Jurafsky and Martin, *Speech and Language Processing* (free draft online): chapters on transformers and large language models.

Resource availability and content change over time; if a link or course has moved, search for the title or ask your mentor.

Check yourself

Why can an LLM state something false with full confidence?

Hint: think about what the model was trained to do (predict a likely next token), what temperature does to its choices, and whether anything in training rewards saying "I don't know". Use your Zentravolin result from step 7 as evidence. Write your answer, then compare it with a peer's.

Discussion prompts (1 hour)

- Explain attention to a non-technical colleague in two minutes, using an analogy.
- Why might a clinical note cost more tokens than an everyday paragraph of the same length? Does that matter?
- Which temperature would you choose for the Note Summariser, and would you ever choose a different one for another feature?
- If the model adds a medication that is not in the note, whose job is it to catch that: the prompt, the code or the clinician?
- What surprised you in today's results?

Daily note template (one page)

- Three things I learned today (my own words).
- One thing that is still unclear, to ask my mentor.
- My answer to the check-yourself question.
- What I will do differently tomorrow.

Definition of done today: notebook runs end to end; token comparison table for medical terms; at least one attention heatmap; low vs high temperature outputs compared in writing; hallucination probe recorded and explained; daily note submitted; questions logged for your mentor.

Looking ahead: Day 4 covers building with LLMs (API calls, prompt engineering, structured JSON output, RAG) with Azure OpenAI, the OpenAI Python SDK and Pydantic, and you write the first piece of the Note Summariser. Before your next shift, ask your mentor for sandbox Azure OpenAI access and install `openai` *and* `pydantic`*. Keep the key in an environment variable, never in your notebook.*